

Trading System Remediation - Week-by-Week Sprint Plan

Week 1: Audit Clarity

Monday

Task: Kick-off + Document State Machine

- ☐ Team meeting: 30min overview of 10 inconsistencies
- ☐ Review current code to identify actual execution states
- ☐ Map current code flow: `CREATE → SEND → ... → FILLED`
- ☐ Identify dead-end states (errors, failures)
- ☐ Create `execution_state_machine.md` draft

Blockers: Need access to production logs to see real state transitions

Deliverable: State machine draft diagram

Tuesday

Task: Complete State Machine + Current Architecture

- ☐ Revise state machine based on code review findings
- ☐ Add timeout/retry behavior
- ☐ Document every state: entry condition, handlers, exit transitions
- ☐ Start `architecture_current_vs_target.md`
 - Diagram current monolithic structure
 - List all components (strategy, execution, broker, dashboard)
 - Identify tight couplings

Blockers: If codebase is too large, extract key flows into separate docs

Deliverable: State machine final + architecture current state section

Wednesday

Task: Target Architecture + Capital Safety Baseline

- ☐ Complete `architecture_current_vs_target.md`
 - Draw target microservices architecture
 - Create gap analysis table (effort estimates)
- ☐ Define capital safety requirements in `capital_safety_requirements.md`
- ☐ Hard kill switch spec

- ☐ Broker reconciliation spec
- ☐ DB audit trail spec
- ☐ Capital allocation spec

Blockers: May need risk/compliance input for safety requirements

Deliverable: Complete target architecture + capital safety baseline

Thursday

Task: Documentation Review + Risk Sign-Off

- ☐ Review all Phase 0 documents with team
- ☐ Walk through each inconsistency: is it resolved in docs?
- ☐ Get sign-off from CTO + risk lead

Blockers: Disagreements on architecture choices

Deliverable: Signed-off documentation

Friday

Task: Phase 1 Planning + Kickoff Preparation

- ☐ Estimate effort for each Phase 1 task (1.1 → 1.4)
- ☐ Create GitHub issues for each task
- ☐ Assign owners
- ☐ Schedule Phase 1 kick-off for Monday

Deliverable: Phase 1 sprint board ready

Week 2: Foundation (Execution + DB)

Monday

Task 1.1 Kickoff: Hard Kill Switch + DB Schema

- ☐ Code review of Phase 0 docs (30min)
- ☐ Task 1.1: Kill switch skeleton
- ☐ Create `trading_system/risk/kill_switch.py`
- ☐ Implement `trigger()` method (idempotent)
- ☐ Add immutable log (file or in-memory)
- ☐ Write unit tests (test idempotency, test logging)
- ☐ Task 1.2: DB schema

- ☐ Create `migrations/001_initial_schema.sql`
- ☐ Design tables: `order_events`, `orders`, `reconciliation`
- ☐ Add indexes for performance
- ☐ Write schema validation tests

Blockers: DB access credentials, migration tool setup (Flyway/Alembic)

Deliverable: Kill switch implementation + schema SQL

Tuesday

Task: Kill Switch Testing + DB Deployment

- ☐ Kill switch: write integration tests
- ☐ Test: trigger halts all trading
- ☐ Test: cannot re-arm via code
- ☐ Test: event logged with timestamp
- ☐ DB: deploy schema to test environment
- ☐ Run migrations
- ☐ Verify tables + indexes created
- ☐ Seed test data (5 sample orders)

Blockers: Test database not accessible, migration failures

Deliverable: Kill switch tested + DB deployed + seed data

Wednesday

Task 1.3 Kickoff: Broker Reconciliation

- ☐ Review broker API docs (what fills endpoint returns)
- ☐ Create `trading_system/broker/reconciliation.py`
- ☐ Implement `find_broker_match(system_order, broker_orders)`
- ☐ Implement `find_system_match(broker_fill, system_orders)`
- ☐ Implement `reconcile(run_date)` main loop
- ☐ Define "minor delta" threshold (\$10, 1 contract)

Blockers: Broker API documentation sparse, need to reverse-engineer from client code

Deliverable: Reconciliation service skeleton

Thursday

Task: Reconciliation Testing + Config Management

- ☐ Reconciliation: write tests
- ☐ Test: orphaned order detection
- ☐ Test: missing fill detection
- ☐ Test: auto-resolve minor deltas
- ☐ Test: alert on major discrepancies
- ☐ Task 1.4: Config management
- ☐ Create `config.py` with dataclass
- ☐ Create `.env.example` template
- ☐ Update all hardcoded values → config reads
- ☐ Add validation on config load

Blockers: Discovering where hardcoded values are hiding (global variables, magic strings)

Deliverable: Reconciliation tested + config externalized

Friday

Task: Phase 1 Review + Phase 2 Planning

- ☐ Demo all Phase 1 deliverables
- ☐ Kill switch: live test (trigger it)
- ☐ DB: show tables + sample data
- ☐ Reconciliation: show test results
- ☐ Config: show .env loading
- ☐ Risk sign-off: capital safety baseline met?
- ☐ Plan Phase 2 week-by-week
- ☐ Assign strategy owner (2.1)
- ☐ Assign refactoring owner (2.2)
- ☐ Assign async queue owner (2.3)

Blockers: Risk team unavailable for sign-off

Deliverable: Phase 1 complete + Phase 2 sprint board

Week 3: Consolidation (Persistence + Config)

Monday

Task: DB Integration + Kill Switch Integration

- ☐ Connect execution engine to DB
- ☐ Write order state transitions to order_events table
- ☐ Write failed orders to order_events with error_msg

- ☐ Verify data persists across restarts
- ☐ Integrate kill switch into execution loop
- ☐ Check `kill_switch.is_live()` before each broker call
- ☐ Test: trigger kill switch while order pending
- ☐ Verify: all open orders cancelled

Blockers: DB connection pool exhaustion under load, asyncio/SQLAlchemy compatibility

Deliverable: Execution fully persisted to DB + kill switch integrated

Tuesday-Wednesday

Task: Reconciliation Daily Run + Config Rollout

- ☐ Set up cron job for daily reconciliation
- ☐ `30 16 * * * python -m trading_system.broker.reconciliation`
- ☐ Log results to database
- ☐ Send alerts if mismatches detected
- ☐ Roll out externalized config to all services
- ☐ Update docker-compose to pass .env
- ☐ Test: dev environment with dev config
- ☐ Test: test environment with test config

Blockers: Cron job permission issues, environment variable passing to subprocesses

Deliverable: Daily reconciliation running + config deployed

Thursday

Task: Phase 1 Final Testing + Documentation

- ☐ End-to-end test: execute order → persisted to DB → reconciliation detects it
- ☐ Stress test: kill switch under load (100 pending orders, trigger)
- ☐ Write runbook: `docs/phase_1_operations.md`
- ☐ How to trigger kill switch manually (if automation fails)
- ☐ How to read reconciliation report
- ☐ How to fix a reconciliation mismatch
- ☐ Update `capital_safety_requirements.md` with "COMPLETE" status for all 4 items

Blockers: Bugs discovered late that require refactoring

Deliverable: Phase 1 fully tested + operational runbook

Task: Phase 1 Sign-Off + Phase 2 Kickoff

- ☐ Compliance + risk + CTO sign-off on Phase 1
- ☐ Formal Phase 2 kickoff meeting
- ☐ Strategy evaluation decision (is it ANN, Grid, or Hybrid?)
- ☐ Architecture decomposition plan (which module to refactor first?)
- ☐ Async queue design walkthrough

Deliverable: Phase 1 signed off + Phase 2 sprint begins

Week 4: Strategy & Architecture (2.1 + 2.2 Start)

Monday

Task 2.1: Strategy Evaluation Complete

- ☐ Decision point meeting (30min)
- ☐ Review backtesting results for ANN and Grid
- ☐ Vote: which strategy to implement in Phase 2?
- ☐ Timeline: if new strategy needed, adjust estimate
- ☐ Create `strategy_evaluation.md`
- ☐ Sharpe ratio, max drawdown, other metrics
- ☐ Rationale for choice
- ☐ Feature engineering approach

Blockers: Backtesting infrastructure missing (need simulator)

Deliverable: Strategy decision made + evaluation doc

Tuesday-Thursday

Task 2.2: Begin Modular Refactoring

- ☐ Create target module structure (no changes to monolith yet)
- ☐ `mkdir -p trading_system/{market_data, strategy, execution, broker, risk, persistence}`
- ☐ Create `__init__.py` in each
- ☐ Create abstract base classes (`BaseStrategy`, `BaseBroker`, `BaseExecutor`)
- ☐ Extract strategy module (non-breaking)
- ☐ Create `strategy/base.py` (abstract interface)
- ☐ Port current strategy logic → `strategy/chosen.py`
- ☐ Unit tests for strategy in isolation
- ☐ Extract execution module (non-breaking)

- ☐ Create `execution/state_machine.py` (copy from Phase 0 design)
- ☐ Create `execution/order.py` (order dataclass + validation)
- ☐ Create `execution/executor.py` (send/await logic)

Blockers: Circular dependencies discovered, existing code tightly coupled

Deliverable: Module structure created + strategy extracted + execution drafted

Friday

Task 2.2: Execution Refactoring Complete

- ☐ Finish execution module
- ☐ Integrate state machine from Phase 0 docs
- ☐ Connect to DB (persist state transitions)
- ☐ Connect to kill switch (check before broker call)
- ☐ Unit tests: state machine correctness
- ☐ Main orchestration
- ☐ Create `main.py` that imports all modules
- ☐ Refactor monolith → `main.py` calls modular services
- ☐ Run monolith in parallel with modular version on test data

Blockers: Breaking changes required, existing APIs don't match module boundaries

Deliverable: Modular execution working in parallel with monolith

Week 5: Async Queue (2.3) + Strategy Impl (2.1)

Monday-Wednesday

Task 2.3: Async Execution Queue

- ☐ Create `execution/queue.py`
- ☐ `ExecutionQueue` class with worker pool (4 workers)
- ☐ Async enqueue, execute, retry with exponential backoff
- ☐ Dead-letter queue for unrecoverable failures
- ☐ Unit tests: enqueue, timeout, retry behavior
- ☐ Integration with executor
- ☐ Replace sync broker calls with `queue.enqueue()`
- ☐ Main loop no longer blocks on broker response
- ☐ Tests: queue under load (100+ orders/second)

Blockers: Asyncio event loop conflicts with existing sync code, broker API latency surprises

Deliverable: Async queue tested + integrated

Thursday-Friday

Task 2.1: Chosen Strategy Implementation

- ☐ Implement chosen strategy (ANN / Grid / Hybrid)
- ☐ Code in `strategy/chosen.py`
- ☐ Feature engineering (as documented in 2.1 decision)
- ☐ Backtest on 2023-2024 data
- ☐ Report: Sharpe, max drawdown, win rate
- ☐ Paper trading (no real capital)
- ☐ Run strategy live on paper account (1 week)
- ☐ Compare paper PnL to backtest
- ☐ Tune if needed

Blockers: Backtest data missing/corrupted, paper account API issues

Deliverable: Strategy implemented + paper traded

Week 6: Integration & Mapping (2.2 finish + 2.4)

Monday-Wednesday

Task 2.2: Full Modular Migration

- ☐ Retire monolith → all traffic through modular main.py
- ☐ Run both versions in parallel (shadowing)
- ☐ Compare outputs (must be identical)
- ☐ Switch traffic (no downtime migration)
- ☐ Finalize module separation
- ☐ Circular dependency audit
- ☐ Each module can be tested independently
- ☐ No global state

Blockers: Traffic switch reveals subtle bugs (state desync between versions)

Deliverable: Full migration complete, monolith retired

Thursday-Friday

Task 2.4: Inconsistency Mapping Complete

- ☐ Update `trading_system_remediation_plan.md` completion matrix
- ☐ Mark all Phase 2 tasks complete

- ☐ List remaining inconsistencies (if any)
- ☐ Verify: all 10 original issues resolved or in Phase 3
- ☐ Write `docs/phase_2_migration.md` (for team knowledge)
- ☐ What changed and why
- ☐ Key design decisions
- ☐ Performance impact (if any)

Deliverable: Phase 2 complete + inconsistency resolution matrix updated

Week 7: Cloud Deployment (3.1)

Monday-Wednesday

Task 3.1: Cloud Infrastructure

- ☐ AWS/OCI setup (choose one)
- ☐ VPC, security groups
- ☐ RDS PostgreSQL instance (restore prod DB backup)
- ☐ EC2 / Compute Engine instances for services
- ☐ Redis cluster for order state cache
- ☐ CI/CD pipeline
- ☐ GitHub Actions → Docker build → push to ECR/Artifact Registry
- ☐ Deploy to staging
- ☐ Run smoke tests

Blockers: AWS/OCI account setup delays, IAM permission issues

Deliverable: Cloud infrastructure ready, services deployed to staging

Thursday-Friday

Task: Load Testing + Optimization

- ☐ Load test: 1000 orders/hour for 24 hours
- ☐ Monitor: latency P99 (should be <100ms)
- ☐ Monitor: database CPU/memory
- ☐ Monitor: broker API rate limits
- ☐ Optimize if needed
- ☐ DB index tuning
- ☐ Connection pooling
- ☐ Async worker scaling

Blockers: Broker API rate limits hit, hidden N+1 queries discovered

Deliverable: Load testing complete, <100ms P99 latency achieved

Week 8: Dashboard + Final Sign-Off (3.2)

Monday-Wednesday

Task 3.2: WebSocket Dashboard

- ☐ FastAPI WebSocket endpoint for PnL
- ☐ React frontend with real-time updates
- ☐ Open positions table
- ☐ Live PnL chart (1Hz update)
- ☐ Order log / audit trail
- ☐ Connect to production database (read-only queries)

Blockers: WebSocket scaling (1000+ concurrent connections?)

Deliverable: Dashboard live and tested

Thursday

Task: Final Testing + Documentation

- ☐ Integration test: end-to-end order flow
- ☐ Strategy generates signal
- ☐ Execution queue processes
- ☐ Broker responds
- ☐ DB persists
- ☐ Dashboard reflects live
- ☐ Compliance audit
- ☐ Reconciliation running daily ✓
- ☐ Audit trail complete ✓
- ☐ Kill switch operational ✓
- ☐ User documentation
- ☐ How to monitor (dashboard)
- ☐ How to understand reconciliation reports
- ☐ How to scale (add workers, instances)

Blockers: Missing audit requirements discovered last minute

Deliverable: Full testing complete + docs finalized

Friday

Task: Production Sign-Off

- ☐ Risk + Compliance + CTO review
- ☐ Final checklist
- ☐ All 10 inconsistencies resolved ✓
- ☐ Kill switch tested ✓
- ☐ Broker reconciliation running ✓
- ☐ Capital allocation enforced ✓
- ☐ Strategy backtested + paper traded ✓
- ☐ Load test passed ✓
- ☐ Production deployment (if approved)

Deliverable: System ready for production or clearly stated why it's not

Daily Standup Template

Monday-Friday 10:00 AM ET (15 min)

Each owner answers:

1. What was accomplished yesterday?
2. What are you doing today?
3. Any blockers?
4. Any insights to share?

Common blockers to monitor:

- Code review delays (assign 2 reviewers, SLA = 24h)
- DB schema changes (plan ahead, test migrations)
- Broker API issues (alert on 429s, rate limit)
- Backtest data stale (refresh daily)
- Production incidents (post-mortem if trading halted)

Risk Register

Risk	Impact	Likelihood	Mitigation
Broker API changes	Execution breaks	Low	Monitor API docs, maintain changelog
DB migration fails in prod	Data loss	Low	Test migrations 3x before prod

Risk	Impact	Likelihood	Mitigation
Strategy backtest overfit	Paper trading loses	High	Walk-forward validation, re-optimize monthly
Reconciliation discovers large discrepancy	Audit failure	Low	Daily monitoring, auto-escalation to ops
Async queue deadlock	Orders hang	Medium	Watchdog timer, dead-letter queue
Kill switch fails to halt	Catastrophic loss	Very Low	Hardware failsafe (circuit breaker at broker)
Latency > 100ms	Missed fills	Low	Dedicated broker connection, CDN
Team turnover mid-project	Schedule slips	Medium	Pair programming, documentation

Success Metrics (End of Week 8)

✔ Process

- All sprints completed on time or with documented re-planning
- Zero critical production incidents
- 100% test coverage for state machine + kill switch

✔ System

- All 10 inconsistencies resolved
- Orders processed end-to-end within SLA (<100ms)
- Dashboard shows live PnL with <1s latency
- Reconciliation running daily with <5% alert rate

✔ Business

- Ready to scale capital with confidence
- Full audit trail for compliance
- Team can operate system without emergency calls